

SE LAB - 3

NAME : VINAY M RAMPUR

SRN : PES1UG24AM455

CLASS : 5TH AIML - H

1.1 System Requirements

Paste the following:

1. The system shall maintain real-time blood-bank inventory.
2. The system shall monitor the shelf life and expiry of blood components.
3. The system shall process emergency blood requests.
4. The system shall cross-reference emergency requests with compatible available blood inventory.
5. When inventory is insufficient, the system shall identify compatible and eligible donors.
6. Donor matching shall consider the specified 10 km radius.
7. The system shall send emergency notifications to matching eligible donors.

1.2 Main Challenges

Performance

- Emergency requests must be processed quickly.
- Compatible blood inventory must be identified rapidly.
- Donor matching and emergency notifications should happen with minimal delay.
- Inventory updates must remain accurate during simultaneous requests.
- Usability
- Emergency requests should be easy to submit.
- Blood Bank Managers should be able to clearly view inventory status.
- The system should clearly communicate whether blood is available or donor matching is required.
- Security
- Donor information must be protected.
- Only authorized users should be able to manage blood inventory.
- Blood allocation must maintain transactional consistency and prevent the same blood unit from being allocated simultaneously to multiple requests.

2. Architectural Style Analysis

Architecture	Advantages	Disadvantages	Suitability for Blood Bank
Layered Architecture	Clear separation of concerns; easy to understand and maintain	Can introduce performance overhead; components may be tightly coupled; difficult to scale individual components	Medium
Microservices Architecture	Independent scaling; fault isolation; independent deployment; services can be developed separately	Higher operational complexity; network latency; distributed data consistency challenges	High
Client-Server Architecture	Centralized control; simple deployment; easier data consistency	Single point of failure; scalability bottleneck; limited fault tolerance	Medium

2.1 Architecture Selection

Selected Architecture: Microservices Architecture

Microservices Architecture is selected for the Blood Bank Inventory & Emergency Donor Matcher because the system contains several distinct functions such as inventory management, emergency request processing, donor matching, and notification. These functions can operate as independently deployable services and can be scaled separately during periods of high emergency demand.

3. Component Identification

3.1 Emergency Request Service

Handles emergency blood requests submitted by the Emergency Requester and forwards them for availability checking and processing.

3.2 Inventory Management Service

Manages blood-bank inventory, including blood group, component type, quantity, allocation, and expiry information.

3.3 Donor Matching Service

Finds compatible and eligible donors when the available blood inventory is insufficient, including the required geographic matching.

3.4 Donor Management Service

Maintains donor information such as blood group, eligibility status, location, and contact information.

3.5 Notification Service

Sends emergency notifications to compatible eligible donors and records notification status.

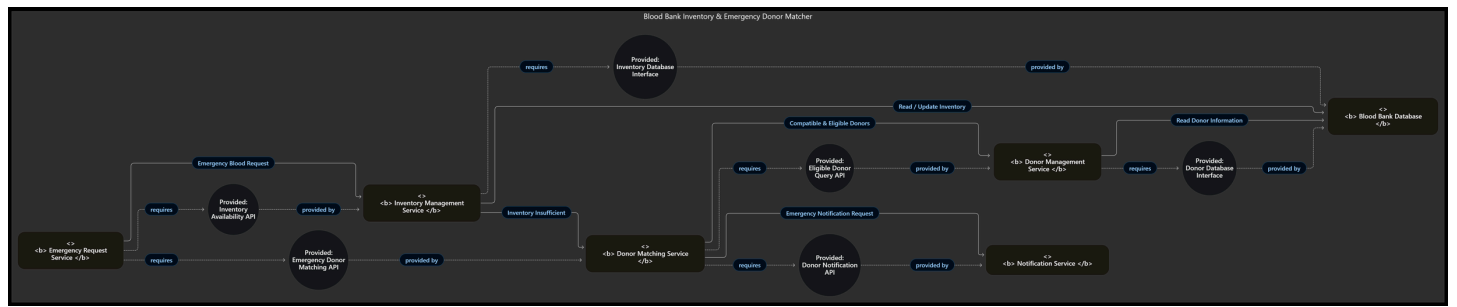
3.6 Blood Bank Database

Stores inventory, donor, emergency request, allocation, and notification data.

3.2 Component Interaction

From Component	To Component	Interface
Emergency Request Service	Inventory Management Service	Inventory Availability API
Emergency Request Service	Donor Matching Service	Emergency Donor Matching API
Donor Matching Service	Donor Management Service	Eligible Donor Query API
Donor Matching Service	Notification Service	Donor Notification API
Inventory Management Service	Blood Bank Database	Inventory Database Interface
Donor Management Service	Blood Bank Database	Donor Database Interface

4.UML DIAGRAM



5. Architectural Justification

Architectural Choice: Microservices Architecture

We selected **Microservices Architecture** for the Blood Bank Inventory & Emergency Donor Matcher because the system consists of several distinct functions that can be separated into independently managed services.

Reason 1 – Independent Functionality:

The system contains separate functions such as emergency request processing, inventory management, donor matching, donor management, and notification. Microservices allow these functions to be developed and maintained as separate services.

Reason 2 – Scalability and Fault Isolation:

Emergency situations may create a sudden increase in requests and donor-matching activity. Individual services such as the Donor Matching Service and Notification Service can be scaled independently without scaling the entire system. Failure in one service can also be isolated from other services.

Security Advantage:

Sensitive donor information can be isolated within the Donor Management Service and accessed through controlled service interfaces. This helps limit direct access to donor information and allows authorization controls to be applied between services.

Performance Advantage:

Services involved in emergency processing, donor matching, and notification can be independently scaled during periods of high demand. This helps the system respond quickly to emergency requests while avoiding unnecessary scaling of unrelated services.